



Variable

Pointer

Declaration

Operations

Example

Exercises

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN - ĐHQG TP HCM

KHOA TOÁN - TIN HỌC

BỘ MÔN ỨNG DỤNG TIN HỌC

CON TRỎ[?]

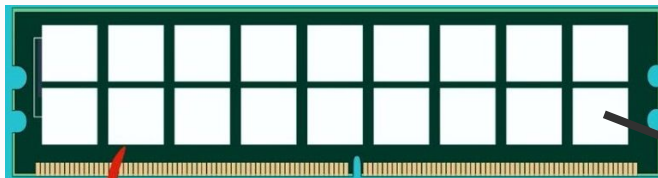
*POINTER

THỰC HÀNH CƠ SỞ LẬP TRÌNH
TUẦN 10 - THÁNG 5 NĂM 2023





BIẾN và vùng nhớ

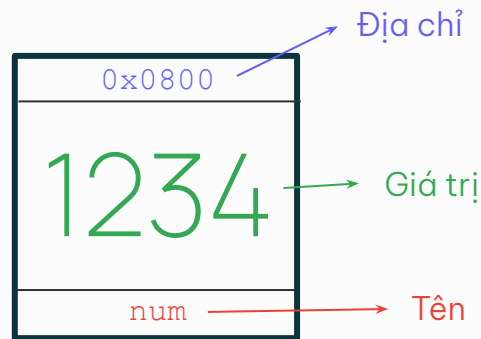


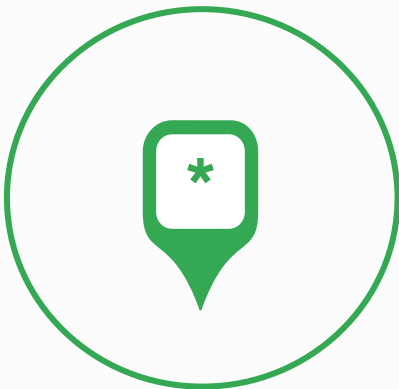
```
int num = 1234;
```

Khi khai báo hoặc khởi tạo một biến, máy tính sẽ thực hiện các công đoạn sau:

1. Khai báo một ô nhớ còn trống trên thanh RAM có **địa chỉ** ngẫu nhiên (0×0800)
2. Gán định danh của ô nhớ là **Tên** mà người dùng đặt (**num**).
3. Gán và lưu trữ **giá trị** vào ô nhớ tuỳ theo kiểu dữ liệu (**1234**).

Đối với mảng, máy tính sẽ cấp phát một dãy ô nhớ có địa chỉ liên tiếp nhau.





THÔNG TIN CHUNG

TÊN: CON TRỎ

LOẠI: BIẾN

GIÁ TRỊ LƯU TRỮ: ĐỊA CHỈ.

XUẤT HIỆN TRONG: MẢNG MỘT CHIỀU, HÀM CON, VÒNG LẶP, ...

CON TRỎ là gì?

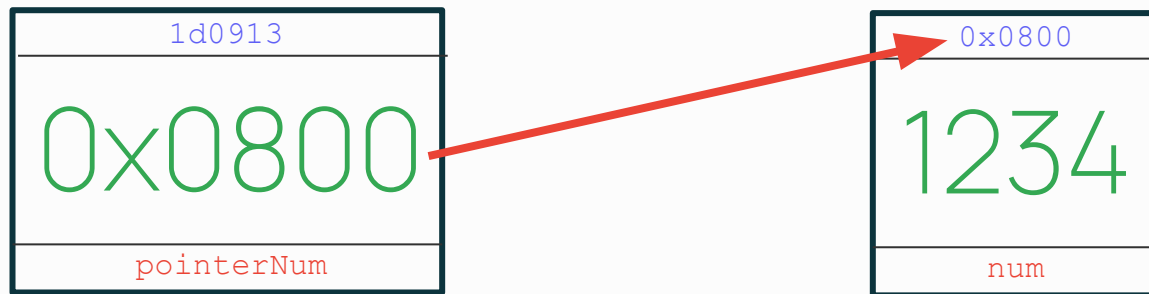
***POINTER** là biến lưu trữ địa chỉ của một vùng nhớ máy tính. Địa chỉ này là của một biến khác.

Con trỏ thường được dùng để:

- Điều chỉnh phù hợp các **đối số** triệu gọi của **hàm con**.
- Hỗ trợ các **cấp phát động**.
- Cải thiện hiệu quả của các cấu trúc định kỳ nhất định. (**mảng**, **chuỗi**, ...)
- Hỗ trợ các **cấu trúc dữ liệu động** như cây nhị phân, danh sách liên kết, ...



Một ***POINTER** chứa **địa chỉ** của một biến khác thì ta gọi ***POINTER** đó đang **trỏ** tới biến đó.



`pointerNum` đang trỏ tới địa chỉ của biến `num`



Cách khai báo biến **CON TRỎ**

```
type *name;
```

```
int *salary;
```

Con trỏ **salary** trỏ tới biến có kiểu số nguyên.

```
float *rating;
```

Con trỏ **rating** trỏ tới biến có kiểu số thực.

```
int array[100];
```

Khai báo một dãy 100 con trỏ có địa chỉ liên kế nhau, với con trỏ ở vị trí đầu tiên (thứ 0) là ***array**

```
char *string;
```

Con trỏ **string** đang trỏ tới biến mang kiểu ký tự.
(chuỗi ký tự)



Toán tử dùng cho **CON TRỎ**



: Trỏ tới địa chỉ mà **con trỏ** đang lưu trữ và lấy giá trị tại đó.



: Lấy địa chỉ của biến.

```
int num = 1234, number;  
int *pointerNum = &num;  
number = *pointerNum + 2;
```

0x0800

1234

num

1d0913

0x0800

*pointerNum

1e2800

1236

number

- Khởi tạo int num và gán 1234, khai báo int number.
- Khởi tạo con trỏ *pointerNum và gán địa chỉ của biến num.
- $number = *(0x0800) + 2$
 $= 1234 + 2 = 1236;$



VÍ DỤ 1: GÁN

```
#include <stdio.h>
int main() {
    int num = 99;
    int *pA, *pB;
    pA = &num;
    pB = pA;
    printf("&num = %p \n", &num);
    printf("pA = %p \n", pA);
    printf("pB = %p \n", pB);
    printf("num = %d \n", num);
    printf("*pA = %d \n", *pA);
    printf("*pB = %d \n", *pB);
    return 0;
}
```

```
&num = 0x7ffcef4ebe14
pA = 0x7ffcef4ebe14
pB = 0x7ffcef4ebe14
num = 99
*pA = 99
*pB = 99
```

Chú ý:

- Vì cách định danh địa chỉ tùy thuộc vào hệ điều hành, máy chủ nên kết quả địa chỉ có thể khác với ví dụ. Tuy nhiên địa chỉ của `&num`, `pA` và `pB` là giống nhau.
- Ở ví dụ này, kết quả địa chỉ được lấy từ <https://www.onlinegdb.com/>



VÍ DỤ 2: NHẬP - XUẤT

```
#include <stdio.h>
int main() {
    float num;
    float *pointerNum;
    pointerNum = &num;
    printf("Nhap num: ");
    scanf("%f", &num);
    printf("num = %f \n", num);
    printf("Nhap num lan nua: ");
    scanf("%f", pointerNum);
    printf("num = %f ", num);
    return 0;
}
```

```
Nhap num: 12
num = 12.000000
Nhap num lan nua: 13
num = 13.000000
```

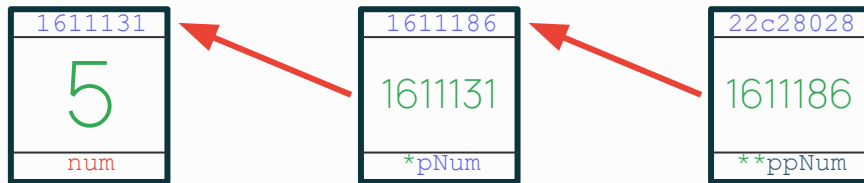
Chú ý:

- Khi ta thay đổi giá trị tại địa chỉ của con trỏ đang trỏ tới thì biến bị trỏ cũng bị thay đổi giá trị.
- Nói cách khác:
`*pointerNum == num`
`pointerNum == &num`



VÍ DỤ 3: CON TRỎ CẤP HAI

```
#include <stdio.h>
int main(){
    int num, *pNum, **ppNum;
    pNum = &num;
    ppNum = &pNum;
    printf("Nhap num: ");
    scanf("%d", *ppNum);
    printf("num = %d", num);
    return 0;
}
```



Chú ý:

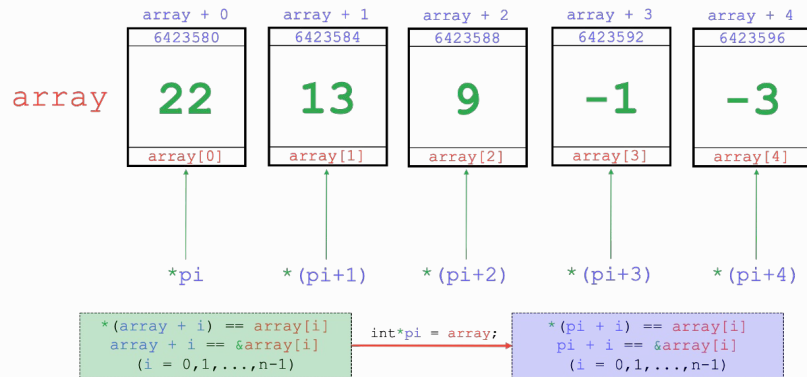
- Con trỏ cấp hai là **con trỏ** trỏ đến địa chỉ của một con trỏ khác.
- Nói cách khác:
$$\begin{aligned} ppNum &== \&pnum \\ *ppNum &== pnum == \&num \\ **ppNum &== num \end{aligned}$$
- Con trỏ cấp 2 thường dùng để thao tác mảng hai chiều và tham trị một biến kiểu con trỏ vào hàm.



VÍ DỤ 4: CON TRỎ TRONG MẢNG 1 CHIỀU

```
#include <stdio.h>

int main()
{
    int array[] = {22,13,9,-1,-3};
    printf("array = %p \n", array);
    printf("*array = %d \n", *array);
    printf("array + 1 = %p \n", array+1);
    printf("* (array + 1) = %d \n",
        *(array+1));
    for (int *pi = array; pi <= array+4; pi++)
        printf("pi = %p ; *pi = %d
        \n", pi, *pi);
    return 0;
}
```



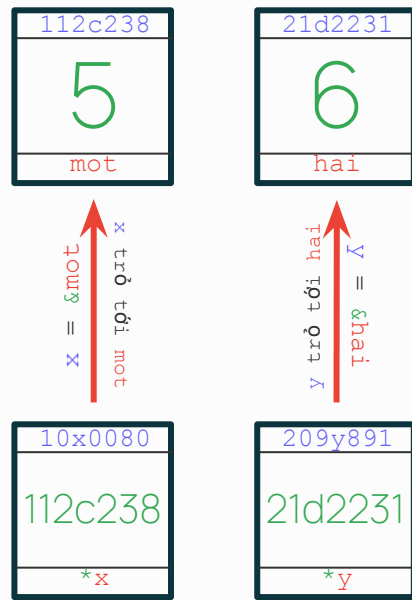
Chú ý:

- Phép toán (+) và (-) chỉ áp dụng cho con trỏ kiểu nguyên. Dùng toán tử tăng giảm con trỏ để thay đổi địa chỉ mà con trỏ đó đang trỏ tới.
- Bản chất của mảng là một dãy con trỏ tuần tự.
- Ta có thể gán mảng cho 1 con trỏ khác. Con trỏ đó sẽ mang địa chỉ của phần tử đầu tiên trong mảng.



VÍ DỤ 5: TRUYỀN THAM TRỊ TRONG HÀM

```
#include <stdio.h>
void swap(int*x,int*y)
{
    int temp;
    temp = *x;
    *x = *y;
    *y = temp;
}
int main()
{
    int mot, hai;
    printf("Nhap so thu nhat: "); scanf(" %d",&mot);
    printf("Nhap so thu hai: "); scanf(" %d",&hai);
    printf("Truoc khi doi cho: (x;y) = ( %d;%d) \n",mot, hai);
    swap(&mot,&hai);
    printf("Sau khi doi cho: (x;y) = ( %d;%d) \n",mot, hai);
    return 0;
}
```



Truyền địa chỉ của **mot** và **hai** vào hàm **swap**, gán lần lượt cho con trỏ ***x** và ***y**.



BÀI TẬP TẠI LỚP

1. Gõ đoạn code dưới vào IDE, compile và trả lời các câu hỏi sau:

```
#include <stdio.h>
int main(){
    int studentID;
    int *pointer = &studentID;
    printf("Nhap ma so sinh vien: ");
    scanf("%d", pointer);
    *pointer += studentID * *pointer;
    printf("So tien hoc phi phai dong la: %d", studentID);
    return 0;
}
```

- a) Ý nghĩa của dấu `*` trong `"*pointer"` ở dòng 4 và dòng 7 là giống nhau hay khác nhau? Vì sao?
- b) Vì sao ở đằng trước `"pointer"` ở dòng 6 không có dấu `&`?
- c) Giải thích dòng 7.



BÀI TẬP TẠI LỚP

2. Viết chương trình khai báo hai biến con trỏ kiểu nguyên `*pointerA` và `*pointerB` sao cho:

- a) Người dùng nhập hai số nguyên bất kỳ từ bàn phím, gán vào hai biến A và B. Thiết lập hai con trỏ sao cho `*pointerA` trỏ đến A, `*pointerB` trỏ đến B.
- b) Xuất ra địa chỉ của `*pointerA` và `*pointerB` đang lưu trữ và giá trị nơi nó trỏ đến.
- c) Khởi tạo biến con trỏ `*pointerC` và gán nó bởi giá trị của `*pointerA`. Xuất ra địa chỉ của `*pointerC` đang lưu trữ và giá trị nơi nó trỏ đến.



BÀI TẬP TỰ LUYỆN

3. Khởi tạo hai biến `a = 12`; `b = 25` và khai báo hai biến con trỏ `*pa` và `*pb`. Viết chương trình để thực hiện các yêu cầu sau:

- a) Gán địa chỉ của biến `a` cho `pa`, địa chỉ của biến `b` cho `pb` rồi xuất giá trị của `pa` và `pb`.
- b) Thêm vào hai câu lệnh `*pa += *pb`; `*pb *= *pa`; xuất ra giá trị của `a` và `b` lúc này.
- c) Giải thích câu lệnh: `*pa = (*pa)*(*pa)*(*pa)`;



4. Gõ 2 đoạn chương trình sau đây vào IDE, debug từng dòng để nghiên cứu cách vận hành.

```
#include <stdio.h>
int main()
{
    int arr[100];
    int n;
    do {
        printf("Nhap so luong phan tu: ");
        scanf("%d", &n);
    } while (n < 1);

    //Nhap mảng
    for (int i = 0; i < n; i++) {
        printf("Nhap a[%d] = ", i);
        scanf("%d", (arr + i));
    }

    //Xuất mảng
    for (int i = 0; i < n; i++) {
        printf("\n Gia tri a[%d] = %d", i, *(arr + i));
    }
    return 0;
}
```

```
#include <stdio.h>
#include <math.h>
//Hàm tính chuẩn Euclide
float nomEuclide(float *x, int n){
    int module = 0;
    for (int i = 0; i < n; i++)
        module += *(x+i)**(x+i);
    return sqrt(module);
}

void main(){
    float x[1000]; int n;
    printf("Nhap so chieu n = ");
    scanf("%d",&n);
    for (int i = 0; i<n; i++){
        printf("Nhap x(%d) = ",i);
        scanf("%f",x+i);
    }
    printf("||x|| = %f",nomEuclide(x,n));
    //truyền mảng x và giá trị n vào hàm
}
```

ĐỌC THÊM

ỨNG DỤNG CON TRỎ: CẤP PHÁT ĐỘNG

[In05 - LẬP TRÌNH C \(P4\) - Con trỏ](#) trang 32



THE END

